UNICA
UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

sAIfer Lab
Joint lab on Safety and Security of AI

# PyTorch Introduction and Automatic Differentiation


**Maura Pintor**
Ricercatrice @ Università di Cagliari
maura.pintor@unica.it

# What is PyTorch?

An open-source Python library for deep learning, built around two ideas:

- Tensors multi-dimensional arrays (like NumPy) but with GPU support.
- Autograd automatic differentiation: PyTorch records every operation and computes gradients automatically.

## Why we use it

- Python API, models look like ordinary classes and functions.
- Building blocks for deep learning: `torch.nn` layers, `torch.optim` optimizers, datasets, transforms.
- Now widely used in research and increasingly in production.

Install: `pip install torch` (already pre-installed in Google Colab)

# The Regression Task

Given pairs $(x, y)$, learn parameters so that the line fits the data.

Model. $\hat{y} = w\,x + b$

Loss (MSE). $L = (1/N) \sum (\hat{y} - y)^2$

Parameters. find $w$ and $b$ that minimize $L$.

In the lab: sklearn.datasets.make_regression → 1000 noisy points, normalised to [0, 1].

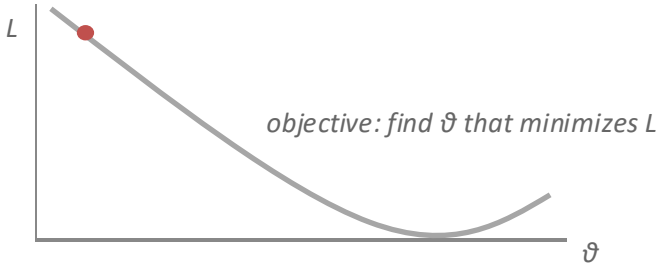
# Finding the Minimum: Gradient Descent

The gradient $\nabla L$ points in the direction of steepest **increase**. The gradient descent steps in the opposite way.

$$\theta \leftarrow \theta - \alpha \nabla L(\theta)$$

- **$\alpha$ (learning rate):** too small → slow; too large → oscillation or divergence.
- **Repeat:** compute loss → compute gradients → update parameters.

For our model the gradients can be derived by hand, but for deep networks they are much more complicated.



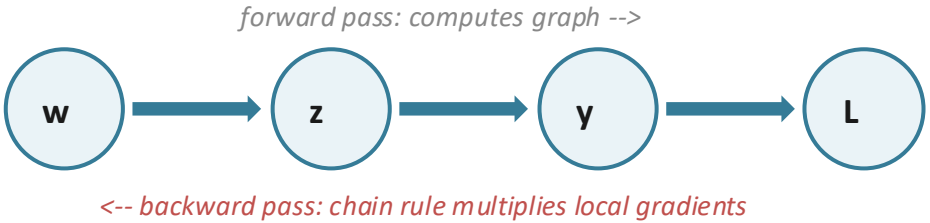
# Chain Rule and Backpropagation

A neural network is a chain of functions. To get the gradient of the loss w.r.t. an early parameter, multiply the local derivatives along the chain.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Example. for $y = (wx + b)^2$, with $z = wx + b$:

$$\frac{\partial y}{\partial z} = 2z \ , \qquad \frac{\partial z}{\partial w} = x \ \rightarrow \ \frac{\partial y}{\partial w} = 2(wx + b)\, x$$



# Automatic Gradients (Autograd)

PyTorch tracks every operation on a tensor flagged **`requires_grad=True`**, then walks the graph backwards on demand.

```
params = torch.tensor([1.0, 0.0], requires_grad=True)
loss   = loss_fn(model(x, *params), y)
loss.backward()       # fills params.grad
print(params.grad)    # tensor([-4.36, 0.25])
```

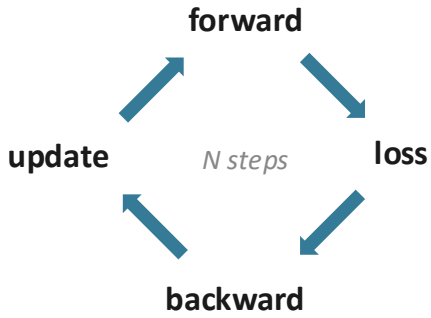
- Same numbers as the hand-derived gradient.
- Gradients **accumulate**: call **`params.grad.zero_()`** after every step.

# The Training Loop

Same four steps batch of samples (in this case we pass the entire dataset).

```
for epoch in range(1, n_epochs+1):
    y_pred = model(x, *params)
    loss   = loss_fn(y_pred, y)
    loss.backward()
    with torch.no_grad():
        params -= lr * params.grad
    params.grad.zero_()
```

The `no_grad` block keeps autograd from tracking the parameter update itself.



# Optimizers and Schedulers

Optimizers replace the manual update step.

```
optimizer = torch.optim.Adam([params], lr=1e-1)
loss.backward();  optimizer.step();  optimizer.zero_grad()
```

- Adam adapts the per-parameter step size with momentum (usually converges faster than plain SGD).

Learning-rate schedulers lower $\alpha$ over iterations (or with other heuristics).

```
scheduler = ReduceLROnPlateau(optimizer, 'min')
scheduler.step(loss)   # called once per epoch
```

- This particular optimizer drops the learning rate when the loss stops improving to help fine-tune near the minimum.

# From Manual Model to torch.nn

PyTorch ships ready-made layers and losses

```
model   = torch.nn.Linear(1, 1)         # in_features, out_features
loss_fn = torch.nn.MSELoss()
opt     = Adam(model.parameters(), lr=1e-1)
```

- `model.parameters()` hands every learnable tensor to the optimizer.
- Call `model(x)` for the forward pass (input shape `(N, 1)`, hence the `.unsqueeze(1)` in the lab).
- The training loop is unchanged, only the model and loss objects are modified.

# Deep Networks: Stacking Layers

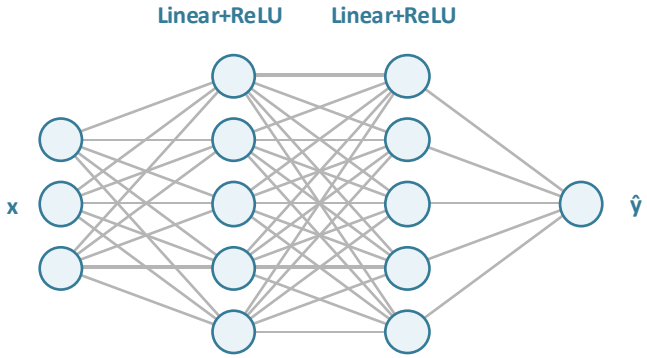
A single Linear layer can only fit a line. Stacking Linear + non-linearity (ReLU) lets the network model learn more difficult curves.

## Quick way: nn.Sequential

```
dnn = nn.Sequential(
    nn.Linear(1, 100), nn.ReLU(),
    nn.Linear(100, 100), nn.ReLU(),
    nn.Linear(100, 1))
```

## Flexible way: subclass nn.Module

```
class NeuralNetwork(nn.Module):
    def __init__(self): ...    # layers
    def forward(self, x): ...  # forward pass
```




*a fully-connected DNN*

# From Regression to Classification

The same DNN can predict a class instead of a number. Three things change: output size, loss, and how we read predictions.

**Output layer.** one neuron per class instead of one scalar.

```
nn.Linear(10, 1)    # regression: one value
nn.Linear(10, C)    # classification: one logit per class
```

**Loss.** swap MSE for `nn.CrossEntropyLoss()`. It applies softmax internally and compares against the integer class label.

- $\text{softmax}(z)_i = \exp(z_i) / \Sigma_j \exp(z_j)$ turns logits into a probability over the C classes.

**Prediction.** pick the class with the highest score: `y_pred = logits.argmax(dim=1)`.

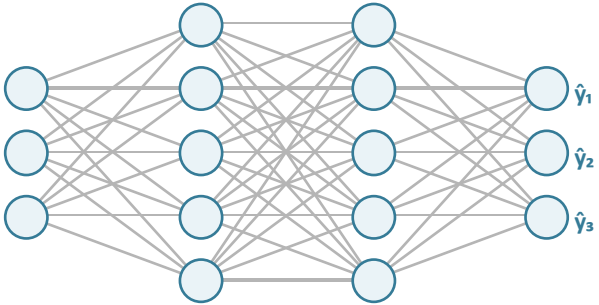
# DNN for Classification: C Output Neurons

Same hidden layers as before. The final Linear layer produces C logits, one per class.

## Forward pass

- Hidden layers extract features from input x.
- Output layer returns logits z, one per class.
- softmax(z) gives class probabilities; argmax picks the predicted class.

```
logits = model(x)              # shape (N, C)
probs  = logits.softmax(dim=1)
y_pred = logits.argmax(dim=1)
```



# Training and Evaluating the Classifier

## Training loop

```
model = NeuralNetwork()
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for epoch in range(epochs):
    logits = model(x)       # (N, C)
    loss = loss_fn(logits, y)   # y is int class labels
    opt.zero_grad(); loss.backward(); opt.step()
```

## Evaluation

- Call `model.eval()` and wrap inference in `torch.no_grad()` to disable gradients.
- Accuracy: fraction of correctly predicted classes.

```
acc = (logits.argmax(dim=1) == y).float().mean()
```

# Image Processing

A fully connected flattens the image and connects every pixel to every neuron

The model has no idea that nearby pixels matter more than distant ones

- **Locality.** Each output should look at a small neighborhood, not the whole image.
- **Translation invariance.** An edge should be detected wherever it appears and in all directions.
- **Parameter sharing.** The same small filter slides across every position, instead of a separate weight per pixel.

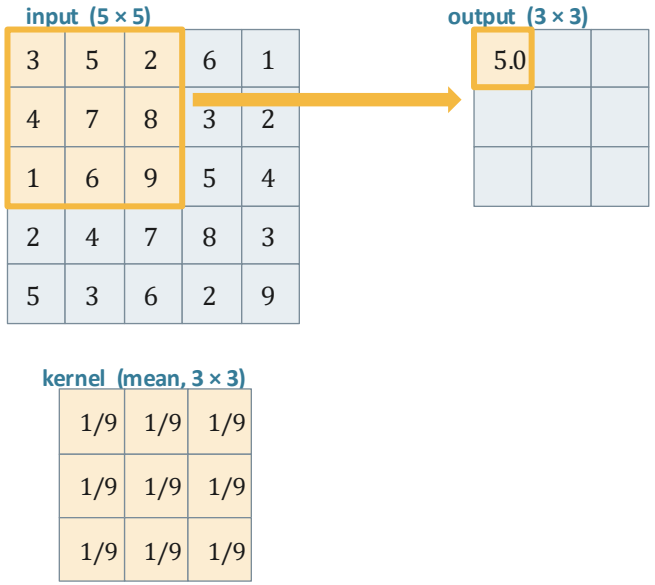
# The Discrete Convolution

A small weight matrix (the **kernel**) slides over the image. At every position we take the elementwise product with the underlying neighborhood and sum.

$$y[i, j] = \Sigma_{u,v}\, k[u, v] \cdot x[i+u, j+v]$$

- **Mean filter** (3 × 3, all entries 1/9): blurs the image.
- **Sobel, edge, Gaussian** are other classic hand-designed kernels.

**Idea of a CNN:** use **learned** kernel weights.

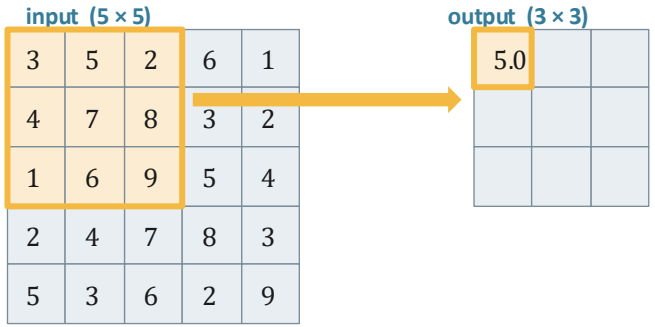


# Convolution Parameters

- **Kernel size** $K$: how big the neighborhood is (commonly 3 or 5).
- **Stride** $S$: how far the kernel moves between applications (1 keeps size, 2 halves it).
- **Padding** $P$: zeros added around the border so the kernel can also center on edge pixels.

Output size for input $I$:

```
nn.Conv2d(in_channels=3,
out_channels=16, kernel_size=3,
padding=1)
```




kernel (mean, 3 × 3)

| | | |
|---|---|---|
| 1/9 | 1/9 | 1/9 |
| 1/9 | 1/9 | 1/9 |
| 1/9 | 1/9 | 1/9 |

$$(3+5+2+4+7+8+1+6+9) \,/\, 9 = 5.0$$

# Building a ConvNet

A ConvNet stacks three repeating ingredients, then ends with a fully connected head for the class scores.

- **Convolution.** learns local filters that produce feature maps.
- **Activation.** ReLU or Tanh, applied elementwise so the model can fit nonlinear patterns.
- **Pooling.** `MaxPool2d(2)` halves the spatial size, keeping the strongest response in each 2 × 2 patch.

The pattern repeats in what are called convolutional blocks, then the high-level representation is attached to a fully connected network

# A first ConvNet for CIFAR-10

CIFAR-10: 32 × 32 RGB images, 10 classes. Two conv blocks shrink space and grow channels, then a small MLP outputs logits.

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)   # 32×32×16
        self.pool1 = nn.MaxPool2d(2)                              # 16×16×16
        self.conv2 = nn.Conv2d(16, 8, kernel_size=3, padding=1)   # 16×16×8
        self.pool2 = nn.MaxPool2d(2)                              # 8×8×8
        self.fc1   = nn.Linear(8 * 8 * 8, 32)
        self.fc2   = nn.Linear(32, 10)                            # 10 logits
    def forward(self, x):
        x = self.pool1(torch.tanh(self.conv1(x)))
        x = self.pool2(torch.tanh(self.conv2(x)))
        x = x.view(-1, 8 * 8 * 8)                          # flatten
        return self.fc2(torch.tanh(self.fc1(x)))
```

# Training on CIFAR-10

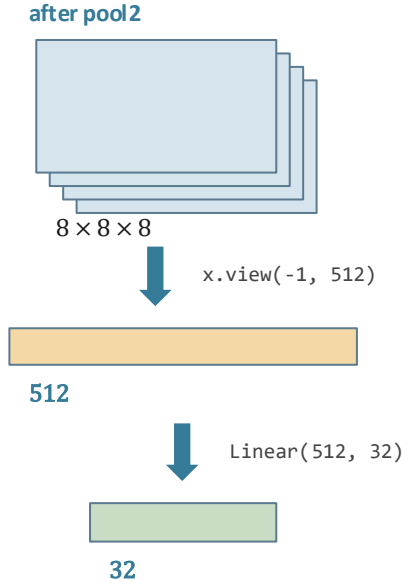
Same loop as before, with batches and a classification loss.

## Setup

```
transf = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465),
                         (0.2023, 0.1994, 0.2010))])
train_loader = DataLoader(cifar_train, batch_size=64, shuffle=True)
net       = Net().to(device)
optimizer = torch.optim.SGD(net.parameters(), lr=1e-2)
scheduler = ReduceLROnPlateau(optimizer)
loss_fn   = nn.CrossEntropyLoss()
```

## Loop

```
for epoch in range(epochs):
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        loss = loss_fn(net(x), y)
        optimizer.zero_grad();  loss.backward();  optimizer.step()
    scheduler.step(loss)
```



# Saving, loading, finetuning

**Save and reload** store `state_dict`, not the model object.

```
torch.save(net.state_dict(), 'cifar_model.pt')
new_model = Net()
new_model.load_state_dict(torch.load('cifar_model.pt'))
new_model.eval()
```

**Finetuning a pretrained model** start from torchvision weights, replace the head, train.

```
from torchvision.models import resnet18, ResNet18_Weights
model = resnet18(weights=ResNet18_Weights.DEFAULT)
for p in model.parameters(): p.requires_grad = False  # freeze backbone
model.fc = nn.Linear(model.fc.in_features, 10)    # new head
opt = torch.optim.SGD(model.fc.parameters(), lr=1e-2)
```

- Train only the head if data is small; unfreeze later layers if you have more.
- Match the input transform that the pretrained model expects (size, normalization).